

PYTHON

Module 13: Operators & Precedence

CODING

Introduction to Operators

Operators are the symbols that perform computations on data. In the CodeHive ecosystem, we classify these into seven distinct functional groups to manage logic and arithmetic effectively.

From simple addition (+) to complex bitwise manipulation, operators are the building blocks of every Python expression.

Arithmetic & Division

Standard math operators like `+`, `-`, `*`, and `**` (exponent) follow algebraic rules.

Python's Dual Division:

- **True Division**

(`/`)******: Always returns a float (e.g., `10 / 2` is `5.0`).

- **Floor Division (`//`)******: Rounds down to the nearest whole number (e.g., `10 // 3` is `3`).**

- **Modulus (`%`)******: Returns the remainder of a division (e.g., `15 % 4` is `3`).**

Assignment & The Walrus

Assignment operators (=) store values. Compound assignments like += and *= update a variable in place.

****The Walrus Operator**
(:=)**: Introduced in Python 3.8, it allows you to assign a value to a variable **inside** an expression. Useful for optimizing if statements and while loops.

Example:
if (n := len(data)) > 10: assigns length to n and checks the condition simultaneously.

Comparison & Chaining

Comparison operators (`==` , `!=` , `>` , `<` , `>=` , `<=`) evaluate to a Boolean.

****Logic Chaining****: Python allows mathematically intuitive checks like `1 < x < 10` instead of requiring two separate statements linked by `and` .

Logical & Identity Operators

• **Logical (and , or , not)**: Used to combine conditional checks.

• **Identity (is , is not)**: Crucial distinction– is checks if two variables point to the same **memory address** (object identity), while **==** only checks if the **values** are the same.

Membership & Bitwise

• ****Membership (in , not in)**:** Tests for presence in strings, lists, or dictionaries.

• ****Bitwise (& , | , ^ , ~ , << , >>)**:** Operates on the binary representation of numbers. Essential for low-level systems programming and data compression.

Operator Precedence (BODMAS)

Precedence determines the order of execution.
Parentheses () always have the highest priority.

Order Overview:

1. Parentheses

2. Exponentiation (**)

3. Multiplication/Division (* , / , // , %)

4. Addition/Subtraction (+ , -)

Example: $2 + 3 * 4$ results in 14 , not 20 .

Knowledge Review

At CodeHive, we emphasize understanding 'is' vs '=='.

- `x = [1, 2]; y = [1, 2] ->`
`x == y` is True, but `x is y` is False (different objects in memory).

- `15 % 4` results in 3 (remainder).

- Use parentheses to ensure your logic is explicit and readable for other developers.